

Curso de:

Desarrollo en React JS

Módulo 1:

Nivelación, Javascript, configuración de entorno

Unidad 4:

React inicial



Presentación

React es una **biblioteca de JavaScript para construir interfaces de usuario**, desarrollada originalmente por **Jordan Walke**, ingeniero de Meta (antes Facebook), e introducida por primera vez en 2013. Surgió como una respuesta a las crecientes necesidades de mantener interfaces web interactivas, eficientes y fáciles de escalar, en un contexto donde las soluciones existentes, basadas en manipulaciones directas del DOM, presentaban serias limitaciones en rendimiento y mantenibilidad.

El contexto de su aparición coincidió con una transición clave en el desarrollo web: **el paso de páginas estáticas a aplicaciones web dinámicas**, donde la interfaz no solo debía mostrar contenido, sino reaccionar en tiempo real a eventos, cambios de estado y acciones del usuario. Bajo este panorama, React llegó para resolver este desafío con una propuesta innovadora: **organizar la interfaz como una colección de componentes reutilizables**, encapsulando estructura, lógica y estilo.

Uno de sus aportes más revolucionarios fue el concepto del **Virtual DOM** (*Document Object Model Virtual*), una representación ligera en memoria del árbol de nodos HTML real, que permite optimizar los cambios en la interfaz mediante un proceso eficiente de comparación y actualización (*reconciliation*). En lugar de modificar directamente el DOM cada vez que algo cambia, una operación costosa en términos de rendimiento, React construye una copia

virtual, detecta las diferencias y actualiza **sólo los elementos necesarios**, logrando aplicaciones más rápidas y fluidas.

Desde su lanzamiento, React ha evolucionado constantemente e impulsado una nueva manera de pensar la construcción de interfaces. De hecho, su influencia se extiende más allá de la biblioteca misma y ha dado lugar a un **ecosistema completo de herramientas y buenas prácticas**, como React Router para navegación, Redux y Context API para manejo de estados, *hooks* para lógica reutilizable, y *frameworks* como Next.js para aplicaciones *fullstack* basadas en React.

En la actualidad, React se utiliza en miles de aplicaciones web y móviles de alta demanda, incluyendo plataformas de redes sociales, comercio electrónico, servicios financieros, herramientas educativas, *dashboards* empresariales, entre muchas otras. Incluso, empresas como Netflix, Airbnb, Instagram y WhatsApp utilizan React en sus interfaces principales, lo que demuestra su capacidad de escalar a nivel global.

En términos de alcance, React permite desarrollar tanto aplicaciones de una sola página (SPAs) como aplicaciones complejas multipágina, progresivas (PWAs) e incluso móviles, mediante herramientas como React Native. Además, su adopción por la comunidad *open source* ha favorecido la creación de cientos de bibliotecas complementarias, patrones de diseño y recursos educativos.

En este contexto, esta unidad se propone brindar una **introducción sólida y estructurada a React**, orientada a comprender sus fundamentos técnicos, sus ventajas frente a otras tecnologías, y su forma particular de estructurar la lógica de interfaz. En detalle, se explorarán sus conceptos clave: los **componentes como unidad fundamental de desarrollo**, la sintaxis JSX que combina

JavaScript con HTML de forma declarativa, y el uso de **props** para transmitir información entre componentes de forma clara y reutilizable.

Además, se abordará la configuración inicial del entorno de desarrollo utilizando herramientas modernas como **Vite** o **Create React App**, y se explicará cómo se organiza típicamente un proyecto en React, incluyendo la gestión de estilos, estructuras de carpetas y convenciones recomendadas.



Objetivos

Que los participantes logren...

- **Comprender** el concepto de componentes en React como unidades funcionales y visuales.
- **Aprender** a estructurar un entorno de trabajo profesional utilizando Vite como herramienta moderna de inicialización.
- **Conocer** la sintaxis y el funcionamiento de JSX como extensión declarativa de JavaScript.
- **Aplicar** el paso de datos entre componentes a través del mecanismo de *props*.
- **Reconocer** la importancia del Virtual DOM en el rendimiento y actualización eficiente de la interfaz.
- **Adoptar** buenas prácticas en la organización de directorios, uso de alias, y manejo de estilos CSS.
- **Establecer** las bases necesarias para continuar con el desarrollo de aplicaciones React más complejas, incluyendo estado, eventos, rutas y consumo de datos externos.



Bloques temáticos

1. **¿Qué es React y cómo funciona?, virtual DOM.**
2. **Configuración de entorno de trabajo.**
 - a. Crear un proyecto React con Vite paso a paso.
3. **Arquitectura de directorios y CSS.**
 - a. Estilos: enfoques posibles.
 - b. Frameworks de utilidad (Tailwind CSS, etc.).
4. **Componentes (¿Qué son?, JSX, props).**
 - a. Componentes: ¿Qué son?
 - b. ¿Qué son las *props*?
 - c. JSX y expresiones JavaScript.

¿Qué es React y cómo funciona? Virtual DOM.

React es una **biblioteca declarativa de JavaScript orientada al desarrollo de interfaces de usuario**. Su principal objetivo es facilitar la creación de aplicaciones web reactivas, es decir, capaces de actualizar su contenido dinámicamente en respuesta a los cambios de datos, de forma eficiente y predecible. Aunque a menudo se lo menciona como un *framework*, React se centra exclusivamente en la "vista" (*view*) dentro del patrón de diseño MVC (Modelo-Vista-Controlador), lo que le permite integrarse fácilmente con otras herramientas para gestionar rutas, estados globales o conexiones con servidores.

La necesidad de React surgió de un problema recurrente en aplicaciones web modernas: **la dificultad para manejar interfaces complejas que cambian con frecuencia**. A medida que las aplicaciones web comenzaron a comportarse más como aplicaciones de escritorio, con múltiples interacciones, actualizaciones dinámicas y personalización del contenido, los enfoques tradicionales con JavaScript y manipulación directa del DOM comenzaron a mostrar sus límites: bajo rendimiento, código difícil de mantener, y *bugs* difíciles de rastrear.

Orígenes y evolución.

React fue desarrollado por **Jordan Walke** en Meta (ex Facebook) en 2011 y lanzado como proyecto de código abierto en 2013. Su objetivo inicial era resolver los cuellos de botella que enfrentaba Facebook con la renderización de elementos como la barra de notificaciones o los comentarios en tiempo real y su adopción se expandió rápidamente gracias a su enfoque centrado en el rendimiento, la reutilización y la simplicidad para construir interfaces modulares.

Con el paso del tiempo, React no sólo consolidó su posición en el *frontend*, sino que se convirtió en un **estándar de facto para el desarrollo de interfaces**, adoptado tanto por *startups* como por grandes corporaciones. En esencia, su éxito inspiró una nueva generación de herramientas y *frameworks* que adoptaron su mismo modelo de componentes.

La arquitectura basada en componentes.

En lugar de construir una interfaz como un bloque monolítico, React propone descomponerla en **componentes independientes**. Cada componente es una unidad funcional y visual que se comporta de la siguiente manera:



Por ejemplo, en lugar de crear una lista de tareas mediante manipulación del DOM tradicional, en React se modela la interfaz como una composición de componentes: **ListaDeTareas**, **Tarea**, **Formulario**, etc., cada uno responsable de su lógica y su visualización.

Este enfoque **favorece la reutilización, la separación de responsabilidades, el testing y la colaboración entre equipos**. Además, es escalable, lo que significa que se adapta tanto a pequeñas interfaces como a proyectos empresariales de gran complejidad.

JSX: JavaScript + HTML.

React utiliza una sintaxis especial llamada **JSX (JavaScript XML)** que permite escribir HTML dentro de JavaScript de forma declarativa. Aunque no es obligatorio, JSX se ha convertido en el estándar de facto para trabajar con React por las siguientes razones:

Hace que el código sea más legible y expresivo.

Legibilidad

Permite representar la estructura visual directamente en el mismo archivo del componente.

Estructura visual

Mejora la productividad del desarrollador al evitar el uso de múltiples archivos para separar lógica y vista.

Productividad del desarrollador

Ejemplo:

```
function Saludo(props) {  
  return <h1>Hola, {props.nombre}!</h1>;  
}
```

Este componente retorna una etiqueta HTML (<h1>) con contenido dinámico basado en las *props*. El resultado es un código más limpio y fácil de mantener.

Virtual DOM: el motor invisible.

El **Virtual DOM** es uno de los pilares tecnológicos que diferencia a React de otras soluciones. En términos sencillos, el DOM real (el árbol de elementos que representa una página web en el navegador) es **lento de modificar**, porque implica operaciones costosas en términos de renderizado.

En este contexto, React evita este problema construyendo un **DOM virtual**, una copia ligera en memoria de la estructura de la interfaz. En concreto, cuando algo cambia (por ejemplo, el usuario escribe en un formulario o se actualiza un dato desde una API), React:

1. Crea una nueva versión del Virtual DOM.
2. Compara la nueva versión con la anterior (mediante un algoritmo llamado *diffing*).
3. Determina el conjunto mínimo de cambios necesarios.
4. Aplica estos cambios al DOM real.

Este proceso, conocido como **reconciliación**, permite que React **optimice automáticamente las actualizaciones**, mejorando la velocidad de la aplicación y reduciendo el uso de recursos.

El flujo unidireccional de datos.

Otro concepto fundamental en React es la **unidireccionalidad del flujo de datos**. En detalle, los datos fluyen **desde los componentes padres hacia los hijos** a través de *props*. Esto significa que cada componente sabe exactamente de dónde provienen sus datos, lo que facilita la depuración y evita estados inconsistentes.

```
function Tarjeta(props) {  
  return <div className="tarjeta">{props.titulo}</div>;  
}  
  
<Tarjeta titulo="Bienvenido" />;
```

En este ejemplo, el componente **Tarjeta** no modifica el dato que recibe, solo lo utiliza para renderizar. Si se necesita modificar el estado, ese cambio ocurre en un componente superior y se propaga hacia abajo.

React en el ecosistema profesional.

Actualmente, React se encuentra en el centro del desarrollo web moderno y su adopción masiva en la industria se debe a los siguientes factores:

- Su **curva de aprendizaje accesible**: al estar basado en JavaScript puro, muchos desarrolladores pueden empezar a usarlo sin necesidad de aprender un nuevo lenguaje.
- Su **ecosistema robusto y modular**: existen soluciones especializadas para manejar rutas (**react-router**), estado global (**Redux**, **Zustand**, **Context API**), formularios (**Formik**, **React Hook Form**), y *testing* (**Jest**, **React Testing Library**).
- Su **portabilidad multiplataforma**: React Native permite reutilizar lógica de interfaz para desarrollar aplicaciones móviles nativas con React.
- Su **comunidad activa** y extensa documentación: miles de tutoriales, foros, paquetes y herramientas contribuyen a su aprendizaje y mejora constante.

Impacto formativo.

El aprendizaje de React no solo permite desarrollar interfaces dinámicas, sino también **incorpora un enfoque de programación basado en funciones puras, reutilización, separación de responsabilidades y optimización de recursos;** habilidades clave para cualquier desarrollador web moderno.

En definitiva, dominar su modelo conceptual y su lógica de funcionamiento es imprescindible para continuar con herramientas más avanzadas como *hooks* personalizados, manejo de efectos secundarios, consumo de APIs externas y construcción de aplicaciones *fullstack* integradas con *backend* y bases de datos.

Configuración de entorno de trabajo.

El primer paso práctico para comenzar a trabajar con React es **configurar el entorno de desarrollo**, es decir, preparar las herramientas necesarias para escribir código, compilarlo y ejecutarlo en un navegador moderno. Este proceso ha evolucionado con el tiempo y hoy en día existen herramientas rápidas y eficientes que permiten crear un proyecto en pocos pasos y comenzar a desarrollar de inmediato.

En línea con ello, en este apartado se trabaja con **Vite**, una herramienta de construcción moderna que se ha convertido en la opción recomendada para crear proyectos React debido a su velocidad, simplicidad y compatibilidad con las últimas versiones del ecosistema JavaScript.

¿Por qué Vite y no Create React App?

Durante años, la forma más común de iniciar un proyecto con React fue mediante **Create React App (CRA)**, una herramienta que abstraía la configuración de Webpack y Babel y permitía comenzar rápidamente. Sin embargo, CRA ha quedado **obsoleta y desactualizada**, con tiempos de carga lentos, una arquitectura monolítica y falta de soporte para mejoras modernas.

En contraste, **Vite** es una herramienta de desarrollo construida por Evan You (creador de Vue.js) que se apoya en **ES Modules nativos** y **Rollup** para producción, lo que la hace:

- Mucho más rápida en el arranque del servidor.
- Capaz de aplicar **hot reload** en tiempo real casi instantáneo.
- Ligera, modular y compatible con TypeScript, React, Vue y otros *frameworks*.

- Ampliamente adoptada por la comunidad y oficialmente recomendada por el equipo de React como reemplazo de CRA.

Requisitos previos.

Antes de comenzar a trabajar con React y Vite, es necesario instalar **Node.js**, un entorno de ejecución de JavaScript que permite utilizar herramientas de desarrollo como **npm** (*Node Package Manager*). Específicamente, Node.js permite ejecutar código JavaScript fuera del navegador, y su instalación incluye **npm**, que se utilizará para crear, configurar e instalar dependencias en el proyecto.

Se recomienda instalar la **versión LTS (Long Term Support)** desde el sitio oficial: <https://nodejs.org>

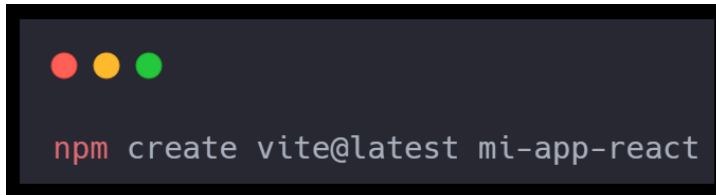
Para verificar que está instalado correctamente se deben correr los siguientes comandos:



Crear un proyecto React con Vite paso a paso.

1. Crear el proyecto con Vite.

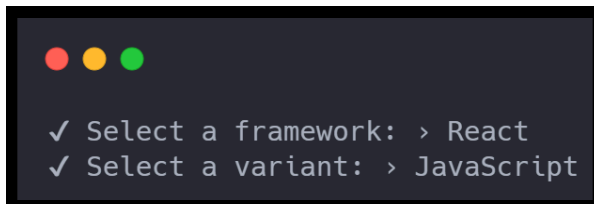
Vite ofrece una herramienta interactiva para crear proyectos de forma rápida. En concreto, se utiliza el siguiente comando para inicializar un nuevo proyecto:



```
npm create vite@latest mi-app-react
```

2. Seleccionar el *framework*.

Este comando descargará e inicializará los archivos base del proyecto. Durante el proceso, Vite preguntará qué *framework* se desea usar. Se debe seleccionar lo siguiente:

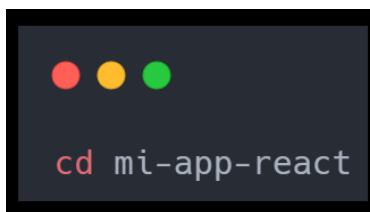


```
✓ Select a framework: > React  
✓ Select a variant: > JavaScript
```

Al finalizar este paso, se habrá creado una carpeta llamada **mi-app-react** con la estructura base de archivos.

3. Ingresar a la carpeta del proyecto.

Después de la creación, se debe ingresar al directorio del nuevo proyecto. Esto se hace con el siguiente comando:

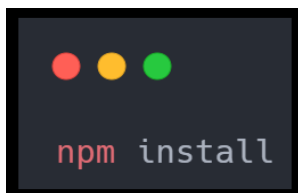


```
cd mi-app-react
```


Una vez dentro, se tendrá acceso a los archivos iniciales y se podrá continuar con la instalación de las dependencias.

4. Instalar las dependencias.

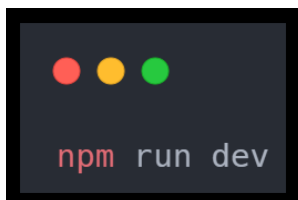
El proyecto creado por Vite contiene un archivo `package.json` que define todas las dependencias del proyecto. Para instalar esas dependencias (incluyendo React y ReactDOM), se ejecuta lo siguiente:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text `npm install` is displayed in a light blue monospace font.

Este comando descarga e instala los paquetes necesarios en la carpeta `node_modules`. No obstante, este paso puede tardar algunos segundos dependiendo de la conexión a internet.

5. Ejecutar el servidor de desarrollo.

Para iniciar la aplicación en modo desarrollo y verla en el navegador, se ejecuta el siguiente comando:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The text `npm run dev` is displayed in a light blue monospace font.

Esto abrirá la app en el navegador en `http://localhost:5173/` o similar. Aunque esta URL puede variar será indicada en la terminal. En general, el servidor de desarrollo de Vite es **extremadamente rápido**, y cualquier cambio en el código se verá reflejado automáticamente en el

navegador gracias a la funcionalidad de *hot module replacement* (HMR), que actualiza los componentes sin recargar la página.

Estructura inicial del proyecto con Vite.

El proyecto creado tendrá una estructura como la siguiente:



Cada uno de estos archivos y carpetas cumple una función específica:

- **index.html**: archivo HTML principal. A diferencia de otros entornos, aquí se encuentra en la raíz del proyecto y contiene el `<div id="root">` donde se monta la aplicación React.
- **vite.config.js**: archivo de configuración de Vite. Permite agregar *plugins*, modificar alias de rutas, configurar el entorno de producción, entre otros.
- **src/**: carpeta que contiene todo el código fuente de la aplicación. Aquí se encuentran los archivos **main.jsx** y **App.jsx**, que son los puntos de entrada de React.

- **public/**: carpeta para colocar archivos estáticos (imágenes, íconos, fuentes) que no necesitan ser procesados por Vite.

Explicación del ciclo de montaje de React.

En **main.jsx** se encuentra el código que monta el componente raíz (**App**) en el DOM:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
)
```

Este archivo realiza tres tareas importantes:

1. **Importa React y ReactDOM**, que son esenciales para crear y renderizar componentes.
2. **Carga el componente raíz App.jsx**, que contiene la estructura principal de la interfaz.
3. **Usa ReactDOM.createRoot** para renderizar el componente dentro del elemento con ID **root** en el **index.html**.

Primer componente en React (**App.jsx**).

El archivo **App.jsx** define la primera vista que el usuario verá al ingresar al sitio. Aquí se puede construir la estructura general de la interfaz:

```
function App() {  
  return (  
    <div className="app">  
      <header>  
        <h1>Mi primer proyecto React</h1>  
      </header>  
      <main>  
        <p>iHola mundo! React está funcionando correctamente.</p>  
      </main>  
    </div>  
  )  
}  
  
export default App
```

Asimismo, este componente puede ir creciendo e incluir otros componentes hijos, rutas, formularios, listas, etc. En la práctica, **App** se convierte en el punto de organización del sistema visual de la aplicación.

Alias de rutas en proyectos React con Vite.

En proyectos React que crecen en complejidad, se vuelve común tener estructuras de carpetas profundas. Esto lleva a rutas de importación largas, difíciles de mantener y propensas a errores. Por ejemplo:

```
import Boton from '../../../components/UI/Boton'
```

Este tipo de rutas relativas:

- Se vuelven frágiles al mover archivos.
- Dificultan la lectura del código.
- No escalan bien en grandes aplicaciones.

Para resolver esto, Vite permite configurar **alias de rutas**, una funcionalidad que permite usar nombres personalizados en lugar de rutas relativas largas. En esencia, esta práctica está ampliamente aceptada en entornos profesionales y mejora la productividad del equipo.

¿Qué es un alias de ruta?

Un **alias** es un atajo para referirse a una carpeta o archivo, de modo que se pueda importar un módulo desde cualquier lugar del proyecto sin preocuparse por la profundidad de la ubicación.

Por ejemplo, en lugar de escribir lo siguiente:

```
import Header from '../../../components/layout/Header'
```

El mismo código se puede escribir de la siguiente manera:

```
import Header from '@components/layout/Header'
```

Allí, @ es un alias que apunta a la carpeta `src/`.

¿Cómo se configura un alias de ruta en Vite?

Editar `vite.config.js`.

Este archivo se encuentra en la raíz del proyecto y se usa para personalizar la configuración del entorno de desarrollo.

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import path from 'path'

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),
    },
  },
})
```

- @ es el alias elegido.
- `path.resolve(__dirname, './src')` indica que @ apunta a la carpeta `src`.
- Usar el alias en cualquier `import`.

```
import LoginForm from '@components/forms/LoginForm'  
import { useAuth } from '@context/AuthContext'  
import logo from '@assets/logo.png'
```

¿Qué ventajas tiene usar alias?



Legibilidad

Los imports son claros y concisos.



Reubicación segura

Se puede mover archivos sin romper las rutas.



Escalabilidad

Es esencial en aplicaciones grandes con muchas carpetas anidadas.



Organización mental

Ayuda a conceptualizar áreas funcionales del proyecto.

Arquitectura de directorios y CSS.

Uno de los aspectos fundamentales en el desarrollo profesional de aplicaciones con React es mantener un **orden claro y escalable en la estructura de carpetas y archivos** del proyecto. De hecho, la forma en que se organiza el código impacta directamente en la mantenibilidad, la colaboración en equipo y la capacidad de extender la aplicación a medida que crece.

En este apartado se aborda la arquitectura base de un proyecto React creado con Vite, junto con las estrategias más comunes para **estructurar componentes, páginas, estilos, contextos, hooks, assets y servicios**, respetando principios de claridad, separación de responsabilidades y escalabilidad. También, se introduce cómo manejar los estilos en React, desde CSS plano hasta enfoques más modernos como módulos, preprocesadores y librerías utilitarias.

Como se mencionó, la estructura inicial es funcional para pequeñas pruebas o prototipos. Sin embargo, a medida que la aplicación crece en tamaño y complejidad, se vuelve fundamental reorganizar los archivos en módulos funcionales y carpetas especializadas para mejorar la mantenibilidad y escalabilidad del código. En esencia, una arquitectura más robusta permite dividir responsabilidades, reducir acoplamientos innecesarios y facilitar tanto la reutilización de componentes como la colaboración en equipo. En este contexto, configurar alias de ruta se vuelve una herramienta clave para simplificar las importaciones, evitando rutas relativas largas y propensas a errores, y permitiendo una navegación más clara y semántica del proyecto.

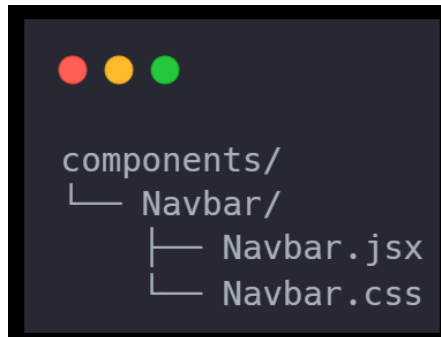

```
src/
├── assets/           # Imágenes, íconos, fuentes
├── components/       # Componentes reutilizables
├── pages/            # Vistas principales del sitio
├── styles/           # Archivos CSS globales o utilitarios
├── context/          # Contextos globales (React Context API)
├── hooks/            # Hooks personalizados
├── services/         # Conexiones con APIs o lógica externa
├── routes/           # Definición de rutas (opcional)
├── utils/            # Funciones auxiliares o helpers
├── App.jsx           # Componente raíz
└── main.jsx          # Punto de entrada
```

Esta organización no es única ni obligatoria, pero es ampliamente utilizada y adaptada según la necesidad del proyecto o equipo. En términos generales, el objetivo es separar responsabilidades, agrupar elementos relacionados y facilitar la navegación por el código.

Componentes.

Los **componentes** son las unidades fundamentales y reutilizables en React. Estas unidades representan fragmentos autónomos e independientes de la interfaz de usuario, encapsulando tanto su estructura visual (HTML/JSX), como su comportamiento (JavaScript) y, en muchos casos, también sus estilos (CSS o preprocesadores). Esta separación modular permite construir interfaces complejas a partir de bloques simples y bien definidos.

No obstante, en proyectos reales, es recomendable organizar cada componente dentro de su propia carpeta dentro de **src/components/**, especialmente si cuenta con lógica propia, estilos particulares, pruebas asociadas o subcomponentes. Por ejemplo, un componente **Button** podría tener la siguiente estructura:



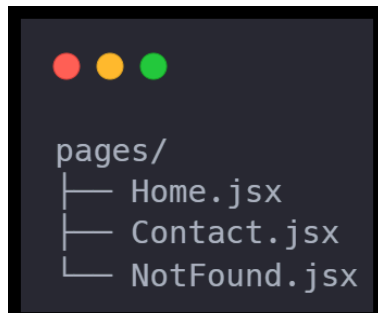
Esta organización promueve la escalabilidad del proyecto, mejora la legibilidad y facilita la reutilización del código. Además, al mantener cada componente aislado, es más fácil depurarlo, testearlo y reemplazarlo sin afectar otras partes de la aplicación. En conjunto con los alias de ruta, esta estructura modular se convierte en una herramienta poderosa para mantener un proyecto limpio, ordenado y profesional.

Páginas (*pages*).

En una aplicación React moderna, especialmente cuando se utiliza React Router para el manejo de rutas, es una práctica común organizar las **páginas o vistas principales** dentro del directorio `src/pages/`. Cada archivo dentro de esta carpeta representa una **pantalla completa** de la aplicación, es decir, una ruta significativa que el usuario puede visitar, como el inicio, el formulario de contacto o una página de error 404.

Este enfoque contribuye a una arquitectura más clara y mantenible, donde cada página actúa como un contenedor que agrupa y estructura diferentes componentes. Por ejemplo, la página `Home.jsx` podría renderizar un carrusel, una sección de productos destacados y un pie de página, todos estos implementados como componentes individuales dentro de `src/components/`.

En general, cuando se implementa junto a React Router, esta organización facilita la definición de rutas limpias y explícitas. Un ejemplo típico de estructura sería el siguiente:



En definitiva, organizar las páginas de esta manera no solo ayuda a mantener una jerarquía clara en el proyecto, sino que también permite escalar con facilidad a medida que la aplicación crece, simplemente agregando nuevas vistas y rutas según sea necesario.

Assets.

La carpeta **assets/** cumple un rol fundamental en cualquier proyecto *frontend*: **almacenar recursos estáticos** que son utilizados por la interfaz visual de la aplicación. Entre estos recursos se incluyen **imágenes, íconos SVG, logos, videos, fuentes personalizadas**, e incluso **archivos de sonido**, dependiendo de las necesidades del proyecto.

Dicha carpeta, ubicada generalmente en **src/assets/**, permite mantener ordenado el código fuente al separar claramente los archivos estáticos del código funcional de la aplicación. Esto mejora la legibilidad y facilita el mantenimiento, ya que todos los recursos visuales están centralizados en un único lugar.

Por ejemplo, si una aplicación utiliza un logo institucional y varias imágenes decorativas o ilustrativas, estas deberían almacenarse aquí:



Desde los componentes de React, estos archivos pueden ser importados de forma directa utilizando alias de ruta, lo que simplifica la escritura de las rutas y mejora la claridad del código:



Esta práctica asegura que los recursos sean **procesados y optimizados automáticamente por Vite** durante el *build* (como minificación de SVG o compresión de imágenes), lo que contribuye al rendimiento y a la calidad final del producto.

Estilos: enfoques posibles.

En React, aplicar estilos a los componentes es una parte clave del desarrollo, ya que permite personalizar la apariencia visual y construir interfaces coherentes, atractivas y funcionales. Para ello, existen diversos **enfoques para trabajar con estilos**, y la elección de uno u otro dependerá de las características del proyecto, el equipo de trabajo y los requisitos técnicos. A continuación se detallan los métodos más comunes y sus principales ventajas.

CSS global (tradicional).

Este método consiste en escribir hojas de estilo convencionales (**.css**) e importarlas a nivel global, generalmente desde el archivo **main.jsx** o **App.jsx**. Este proceso es sencillo y directo, pero puede generar conflictos de clases si no se administra correctamente.

A screenshot of a code editor with a dark background. At the top left, there are three colored circles (red, yellow, green) representing window controls. The code shown is:

```
// main.jsx
import './index.css'
```

Este método es ideal para **prototipos rápidos, pequeños proyectos, o estilos base como resets o tipografías**.

CSS por componente (modular).

Una forma más organizada consiste en crear un archivo `.css` o `.module.css` por cada componente y ubicarlo junto a su archivo JSX. Esta técnica favorece la cohesión del código, especialmente cuando los estilos están directamente relacionados con un único componente.



```
// Button.jsx
import './Button.css'

// o con módulos CSS
import styles from './Button.module.css'
```

Los módulos CSS (`.module.css`) generan nombres de clase únicos automáticamente, evitando conflictos entre estilos.

Esta técnica es ideal para **proyectos medianos o grandes que no usan soluciones con JS-in-CSS**.

Preprocesadores (SASS, SCSS, LESS).

Los **preprocesadores CSS** son herramientas que amplían las funcionalidades nativas de CSS, permitiendo escribir estilos de manera más estructurada, reutilizable y mantenible. Luego, el código se compila a CSS puro para que los navegadores puedan interpretarlo.

- **SASS:** *Syntactically Awesome Stylesheets*. Es uno de los preprocesadores más populares. Introduce variables, anidamiento, *mixins* y herencia,

facilitando la organización de los estilos.

- **SCSS:** Sassy CSS. Es una variante de SASS que utiliza una sintaxis más cercana a la de CSS tradicional, por lo que resulta más intuitiva para quienes ya están familiarizados con el lenguaje.
- **LESS:** *Leaner Style Sheets*. Otro preprocesador que añade características como variables y funciones matemáticas, muy usado inicialmente en entornos como Node.js.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in SCSS and defines a class for a button with padding and a hover effect that darkens the background color.

```
// Button.scss
.button {
  padding: $spacing;
  &:hover {
    background-color: darken($color, 10%);
  }
}
```

Estas herramientas son ideales para **equipos con experiencia en CSS avanzado y necesidad de reutilización**.

CSS-in-JS (*styled-components*, Emotion, etc.).

Esta estrategia permite escribir estilos directamente dentro del archivo de JavaScript usando librerías externas. Por ejemplo, con **styled-components**:

```
import styled from 'styled-components'

const Button = styled.button`
  background: blue;
  color: white;
  padding: 1rem;
`
```

Esta estrategia es ideal para **proyectos en React** (u otros *frameworks* similares) donde se busca encapsular estilos por componente, mantener coherencia entre lógica y presentación, y aprovechar la potencia de JavaScript para generar estilos dinámicos.

Frameworks de utilidad (Tailwind CSS, etc.).

Los **frameworks de utilidad** representan un enfoque moderno y altamente eficiente para aplicar estilos en proyectos web. En lugar de escribir reglas CSS personalizadas o crear clases específicas, estos *frameworks* proporcionan **clases utilitarias predefinidas** que encapsulan estilos específicos y reutilizables. Entre ellos, el más popular y ampliamente adoptado en el ecosistema React es **Tailwind CSS**.

¿Cómo funciona Tailwind?

Tailwind permite escribir estilos directamente en el atributo `className` de los elementos JSX usando una combinación de **clases utilitarias** que representan

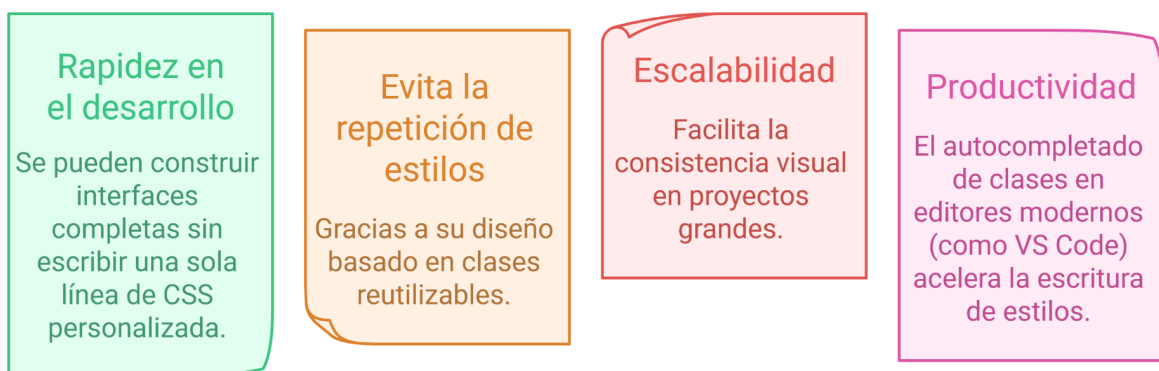
propiedades CSS comunes: colores, tamaños, márgenes, tipografía, sombras, etc.

```
<button className="bg-blue-500 hover:bg-blue-700 text-white py-2 px-4 rounded">
  Enviar
</button>
```

En el ejemplo presentado, el botón aplica:

- **bg-blue-500**: color de fondo azul.
- **hover:bg-blue-700**: color de fondo más oscuro al pasar el *mouse*.
- **text-white**: color de texto blanco.
- **font-bold**: fuente en negrita.
- **py-2 px-4**: padding vertical y horizontal.
- **rounded**: bordes redondeados.

Ventajas de Tailwind y frameworks similares



Componentes (¿Qué son?, JSX, props).

Componentes: ¿Qué son?

React se basa en un concepto fundamental que lo distingue de otras bibliotecas y *frameworks*: los **componentes**.

En el modelo de desarrollo tradicional con HTML, CSS y JavaScript, las interfaces se construyen como una estructura única y global. En cambio, React propone **dividir la interfaz en pequeñas piezas reutilizables, llamadas componentes**, que pueden componerse unas con otras para formar estructuras más complejas.

Este enfoque modular transforma por completo la forma de construir aplicaciones web, ya que permite reutilizar, mantener y escalar la interfaz con mayor facilidad.

¿Qué es un componente en React?

Un **componente** es una unidad funcional y visual que encapsula una parte de la interfaz junto con su lógica y, opcionalmente, su estilo. En particular, cada componente puede comportarse como un bloque independiente, capaz de recibir datos y responder a eventos.

Por ejemplo, un botón, una tarjeta de producto, un encabezado de sección o incluso una página entera pueden ser componentes. Para ello, React permite definir componentes como **funciones JavaScript** que devuelven un fragmento de interfaz escrito en JSX.

Desde una perspectiva técnica, un componente es simplemente una función que retorna un valor JSX. Ese valor será traducido por React en elementos del DOM real que se verán en la página web.

¿Qué es JSX?

JSX (JavaScript XML) es una **extensión de sintaxis para JavaScript** que permite escribir estructuras similares a HTML dentro del código JavaScript. Aunque parece HTML, en realidad JSX es una abstracción que React transforma en llamadas a su propia API (**React.createElement**) para crear nodos virtuales.

Esto hace posible escribir componentes de forma más intuitiva y legible tal como se presenta a continuación:



```
function Saludo() {  
  return <h2>Hola, mundo</h2>;  
}
```

Este código, aunque visualmente se asemeja a HTML, es compilado por herramientas como Babel para convertirse en instrucciones de JavaScript puras. Así, gracias a JSX, se pueden combinar lógica y visualización dentro del mismo archivo, mejorando la expresividad y simplicidad del código.

Estructura básica de un componente.

Un componente funcional mínimo tiene la siguiente forma:

```
function MiComponente() {  
  return (  
    <div>  
      <h1>Título</h1>  
      <p>Este es un componente básico en React.</p>  
    </div>  
  )  
}
```

Este componente, al ser invocado como `<MiComponente />`, renderiza una estructura que contiene un encabezado y un párrafo. En este sentido, la posibilidad de encapsular esta lógica en una función permite reutilizarla en distintos lugares de la aplicación.

Reutilización de componentes.

Una de las principales ventajas de React es que los componentes pueden usarse múltiples veces con diferentes datos. Esto se logra a través del uso de ***props***.

¿Qué son las *props*?

- Las *props* son **de solo lectura**: un componente no debe modificarlas directamente.
- Las *props* permiten **composición**: se pueden combinar múltiples componentes dentro de otros, formando estructuras jerárquicas.
- Se pueden pasar tanto valores primitivos (**string**, **number**, **boolean**) como objetos, funciones o incluso otros componentes.

El siguiente es un ejemplo con función como prop:

```
function Boton({ onClick, texto }) {  
  return <button onClick={onClick}>{texto}</button>;  
}  
  
function App() {  
  const manejarClick = () => alert("¡Click!");  
  return <Boton onClick={manejarClick} texto="Presionar" />;  
}
```

Composición de componentes.

En React, los componentes no solo se definen de forma aislada, sino que se **componen entre sí**, permitiendo construir interfaces cada vez más complejas a partir de piezas simples.

```
function Tarjeta({ titulo, descripcion }) {  
  return (  
    <div className="tarjeta">  
      <h3>{titulo}</h3>  
      <p>{descripcion}</p>  
    </div>  
  );  
}  
  
function App() {  
  return (  
    <div>  
      <Tarjeta titulo="Curso de React" descripcion="Aprendé a crear componentes." />  
      <Tarjeta titulo="Diplomatura Fullstack" descripcion="Todo el desarrollo web en un solo lugar." />  
    </div>  
  );  
}
```

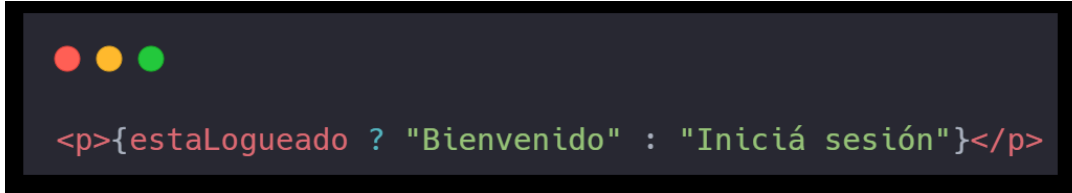
Este ejemplo muestra cómo el mismo componente **Tarjeta** puede reutilizarse con contenido distinto gracias al uso de *props*.

JSX y expresiones JavaScript.

Dentro de JSX, se pueden usar expresiones de JavaScript encerradas entre `{ }`. Esto permite combinar contenido dinámico y lógica dentro de la interfaz:

```
const nombre = "Gabriel";  
  
function Saludo() {  
  return <h2>Hola, {nombre.toUpperCase()}</h2>;  
}
```

También, se pueden usar ternarios o funciones dentro de JSX para controlar qué se muestra:



```
<p>{estaLogueado ? "Bienvenido" : "Iniciá sesión"}</p>
```

Conclusión del bloque.

Los **componentes**, **JSX** y **props** forman el núcleo conceptual de React. Por ello, comprender su funcionamiento permite desarrollar interfaces estructuradas, dinámicas y reutilizables. De esta forma, el paradigma de componentes propone un cambio de mentalidad: en lugar de pensar en páginas completas, se piensa en **pequeñas unidades funcionales que se ensamblan**.

Esta manera de construir interfaces ha sido ampliamente adoptada por la industria y constituye una de las razones principales por las que React se mantiene como una herramienta de referencia en el desarrollo *frontend* moderno.



Bibliografía utilizada y sugerida

JavaScript.info. (2025). *The Modern JavaScript Tutorial*. <https://javascript.info>¹

MDN Web Docs. (s.f.-a). *CSS*. Mozilla Corporation.
<https://developer.mozilla.org/es/docs/Web/CSS>²

MDN Web Docs. (s.f.-b). *JavaScript*. Mozilla Corporation.
<https://developer.mozilla.org/es/docs/Web/JavaScript>³

MDN Web Docs. (s.f.-c). *Modelo de Objetos del Documento (DOM)*. Mozilla Corporation.
[https://developer.mozilla.org/es/docs/Web/API/Document Object Model](https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model)

¹ Recurso complementario para reforzar conceptos previos como funciones, objetos y paso de parámetros, fundamentales para comprender *props*.

² Recurso utilizado para describir los distintos enfoques de uso de CSS en React: archivos globales, módulos, preprocesadores y librerías utilitarias.

³ Base de referencia para conceptos de JavaScript, estructura del DOM, y fundamentos previos necesarios para entender el entorno de React.